

Fine-tuning a Summarization model

ভূমিকা

এই অধ্যায়ে আমরা দেখব কীভাবে Transformer মডেল ব্যবহার করে লম্বা ডকুমেন্টকে সংক্ষিপ্ত সারাংশে রূপান্তরিত করা যায়—এই কাজটাকে বলা হয় **Text Summarization**।

এটি NLP-র সবচেয়ে চ্যালেঞ্জিং কাজগুলোর একটি। কারণ এতে দরকার হয়:

- দীর্ঘ অনুচ্ছেদ বোঝার ক্ষমতা
- মূল বিষয়গুলো ধরে রেখে সংগত ভাষায় লেখার ক্ষমতা

তবে যখন এটি ভালোভাবে কাজ করে, তখন এটি ব্যবসায়িক প্রক্রিয়াগুলোকে দারুণভাবে গতিশীল করে তোলে—বিশেষজ্ঞদের আর পুরো লম্বা ডকুমেন্ট পড়তে হয় না।

এই অধ্যায়ে একটা বিশেষ মজার বিষয় আছে: আমরা ইংরেজি ও স্প্যানিশ—দুটো ভাষার জন্য একটি **bilingual** মডেল তৈরি করব।

শেষে আমাদের মডেল এই ধরনের customer review গুলো summarize করতে পারবে:

“I loved The dash diet weight loss Solution. Never hungry. I would recommend this diet.” → **Summary:** “Easy to follow!!!!”

ধাপ ১—Multilingual Corpus তৈরি

ডেটাসেট নির্বাচন

আমরা **Multilingual Amazon Reviews Corpus** ব্যবহার করব। এই corpus-এ ছয়টি ভাষায় Amazon পণ্যের রিভিউ রয়েছে। প্রতিটি রিভিউর সাথে একটি ছোট **title** আছে—সেই title-গুলোই আমাদের **target summary** হিসেবে কাজ করবে।

প্রথমে Hugging Face Hub থেকে ইংরেজি ও স্প্যানিশ ডেটাসেট লোড করা যাক:

```
from datasets import load_dataset
spanish_dataset = load_dataset("amazon_reviews_multi", "es")
english_dataset = load_dataset("amazon_reviews_multi", "en")
english_dataset
```

আউটপুট:

```
DatasetDict({
  train: Dataset({
```

```

        features: ['review_id', 'product_id', 'reviewer_id', 'stars', 'review_body', 'review_title',
'language', 'product_category'],
        num_rows: 200000
    })
    validation: Dataset({
        features: ['review_id', 'product_id', 'reviewer_id', 'stars', 'review_body', 'review_title',
'language', 'product_category'],
        num_rows: 5000
    })
    test: Dataset({
        features: ['review_id', 'product_id', 'reviewer_id', 'stars', 'review_body', 'review_title',
'language', 'product_category'],
        num_rows: 5000
    })
})

```

দেখা যাচ্ছে, প্রতিটি ভাষার জন্য train split-এ ২ লক্ষ রিভিউ এবং validation ও test split-এ ৫ হাজার করে রিভিউ রয়েছে। আমাদের প্রয়োজনীয় তথ্য **review_body** ও **review_title** কলামে আছে।

ডেটা দেখি একটু

Training set থেকে কয়েকটি random sample দেখার জন্য একটি ছোট function তৈরি করা যাক:

```

def show_samples(dataset, num_samples=3, seed=42):
    sample = dataset["train"].shuffle(seed=seed).select(range(num_samples))
    for example in sample:
        print(f"\n>> Title: {example['review_title']}")
        print(f">> Review: {example['review_body']}")

```

```
show_samples(english_dataset)
```

আউটপুট:

```

'>> Title: Worked in front position, not rear'
'>> Review: 3 stars because these are not rear brakes as stated in the item description. At least
the mount adapter only worked on the front fork of the bike that I got it for.'
'>> Title: meh'
'>> Review: Does it's job and it's gorgeous but mine is falling apart, I had to basically put it
together again with hot glue'
'>> Title: Can't beat these for the money'
'>> Review: Bought this for handling miscellaneous aircraft parts and hanger "stuff" that I
needed to organize; it really fit the bill. The unit arrived quickly, was well packaged and arrived
intact (always a good sign). There are five wall mounts-- three on the top and two on the bottom.
I wanted to mount it on the wall, so all I had to do was to remove the top two layers of plastic
drawers, as well as the bottom corner drawers, place it when I wanted and mark it; I then used

```

some of the new plastic screw in wall anchors (the 50 pound variety) and it easily mounted to the wall. Some have remarked that they wanted dividers for the drawers, and that they made those. Good idea. My application was that I needed something that I can see the contents at about eye level, so I wanted the fuller-sized drawers. I also like that these are the new plastic that doesn't get brittle and split like my older plastic drawers did. I like the all-plastic construction. It's heavy duty enough to hold metal parts, but being made of plastic it's not as heavy as a metal frame, so you can easily mount it to the wall and still load it up with heavy stuff, or light stuff. No problem there. For the money, you can't beat it. Best one of these I've bought to date-- and I've been using some version of these for over forty years.'

Try it out! `Dataset.shuffle()` কমান্ডে *random seed* পরিবর্তন করে আরও রিভিউ দেখুন। যদি আপনি স্প্যানিশ জানেন, তাহলে `spanish_dataset`-এর কিছু রিভিউ দেখুন—*title* গুলো কি সত্যিই ভালো *summary* মনে হচ্ছে?

পণ্যের ক্যাটাগরি বিশ্লেষণ

৪ লক্ষ রিভিউতে একটি GPU-তে ট্রেনিং করতে অনেক সময় লাগবে, তাই আমরা একটি নির্দিষ্ট পণ্যের ক্যাটাগরিতে মনোযোগ দেব। কোন ক্যাটাগরিগুলো পাওয়া যাচ্ছে, সেটা দেখতে `english_dataset` কে `pandas.DataFrame` এ রূপান্তর করা যাক:

```
english_dataset.set_format("pandas")
english_df = english_dataset["train"][:]
# Show counts for top 20 products
english_df["product_category"].value_counts()[:20]
```

আউটপুট:

home	17679
apparel	15951
wireless	15717
other	13418
beauty	12091
drugstore	11730
kitchen	10382
toy	8745
sports	8277
automotive	7506
lawn_and_garden	7327
home_improvement	7136
pet_products	7082
digital_ebook_purchase	6749
pc	6401
electronics	6186
office_product	5521

```
shoes          5197
grocery        4730
book           3756
Name: product_category, dtype: int64
```

Amazon তো বই বেচেই শুরু হয়েছিল! তাই আমরা **book** এবং **digital_ebook_purchase** ক্যাটাগরিতে ফোকাস করব।

বই ফিল্টার করা

```
def filter_books(example):
    return (
        example["product_category"] == "book"
        or example["product_category"] == "digital_ebook_purchase"
    )
```

Filter apply করার আগে **english_dataset**-এর format পুনরায় "arrow"-এ ফিরিয়ে আনি:

```
english_dataset.reset_format()
```

এখন filter function apply করি এবং sample দেখি:

```
spanish_books = spanish_dataset.filter(filter_books)
english_books = english_dataset.filter(filter_books)
show_samples(english_books)
```

আউটপুট:

```
'>> Title: I'm dissapointed.'
```

```
'>> Review: I guess I had higher expectations for this book from the reviews. I really thought I'd at least like it. The plot idea was great. I loved Ash but, it just didnt go anywhere. Most of the book was about their radio show and talking to callers. I wanted the author to dig deeper so we could really get to know the characters. All we know about Grace is that she is attractive looking, Latino and is kind of a brat. I'm dissapointed.'
```

```
'>> Title: Good art, good price, poor design'
```

```
'>> Review: I had gotten the DC Vintage calendar the past two years, but it was on backorder forever this year and I saw they had shrunk the dimensions for no good reason. This one has good art choices but the design has the fold going through the picture, so it's less aesthetically pleasing, especially if you want to keep a picture to hang. For the price, a good calendar'
```

```
'>> Title: Helpful'
```

```
'>> Review: Nearly all the tips useful and. I consider myself an intermediate to advanced user of OneNote. I would highly recommend.'
```

রিভিউগুলো শুধু বইয়ের মধ্যে সীমাবদ্ধ নয়, ক্যালেন্ডার বা OneNote-এর মতো জিনিসও আছে। তবুও domain টা summarization মডেল ট্রেনিংয়ের জন্য যথেষ্ট ভালো।

Bilingual Dataset তৈরি

এখন ইংরেজি ও স্প্যানিশ রিভিউগুলো একটিমাত্র `DatasetDict` object-এ একত্রিত করব। `Datasets`-এর `concatenate_datasets()` function ব্যবহার করে দুটো `Dataset` object পর পর জুড়ে দেওয়া যায়:

```
from datasets import concatenate_datasets, DatasetDict
books_dataset = DatasetDict()
for split in english_books.keys():
    books_dataset[split] = concatenate_datasets(
        [english_books[split], spanish_books[split]]
    )
    books_dataset[split] = books_dataset[split].shuffle(seed=42)
# Peek at a few examples
show_samples(books_dataset)
```

আউটপুট:

```
'>> Title: Easy to follow!!!!'
'>> Review: I loved The dash diet weight loss Solution. Never hungry. I would recommend this
diet. Also the menus are well rounded. Try it. Has lots of the information need thanks.'
'>> Title: PARCIALMENTE DAÑADO'
'>> Review: Me llegó el día que tocaba, junto a otros libros que pedí, pero la caja llegó en mal
estado lo cual dañó las esquinas de los libros porque venían sin protección (forro).'
'>> Title: no lo he podido descargar'
'>> Review: igual que el anterior'
```

চমৎকার! ইংরেজি ও স্প্যানিশ রিভিউয়ের মিশ্রণ দেখা যাচ্ছে।

শব্দ বিতরণ পরীক্ষা ও **Short Title** ফিল্টার

Training corpus প্রস্তুত হওয়ার আগে একটি গুরুত্বপূর্ণ বিষয় দেখতে হবে—রিভিউ ও title-গুলোতে শব্দের বিতরণ কেমন। Summarization task-এ এটি বিশেষভাবে গুরুত্বপূর্ণ, কারণ ডেটায় খুব ছোট reference summary থাকলে মডেল শুধুমাত্র ১-২ শব্দের আউটপুট দিতে শিখে যায়।

Plot দেখলে বোঝা যাবে, title-গুলো বেশিরভাগ ক্ষেত্রে মাত্র ১-২ শব্দের। তাই আমরা খুব ছোট title-গুলো ফিল্টার করে দেব, যাতে মডেল আরও আকর্ষণীয় summary তৈরি করতে শেখে:

```
books_dataset = books_dataset.filter(lambda x: len(x["review_title"].split()) > 2)
```

ধাপ ২—**Text Summarization**-এর জন্য মডেল

Summarization কে Translation-এর মতো ভাবুন

Text summarization আসলে machine translation-এর মতোই একটি কাজ—আমরা একটি রিভিউকে তার সংক্ষিপ্ত version-এ “translate” করছি। সেকারণে Transformer summarization মডেলগুলো বেশিরভাগই **encoder-decoder** আর্কিটেকচার ব্যবহার করে।

নিচের সারণিতে কিছু জনপ্রিয় pretrained মডেল দেখানো হলো:

Transformer মডেল বিবরণ Multilingual?

GPT-2 Auto-regressive language model। Input-এর শেষে “TL;DR” জুড়ে দিলে summary তৈরি করতে পারে।

✗ **PEGASUS** Multi-sentence text-এ masked sentence predict করার pretraining objective ব্যবহার করে। Popular benchmark-এ উচ্চ স্কোর পায়।

✗ **T5** সব কাজ text-to-text framework-এ সম্পন্ন করে। Document summarize করতে input format: `summarize: ARTICLE`।

✗ **mT5** T5-এর multilingual version। 101টি ভাষায় multilingual Common Crawl corpus (mC4)-এ pretrained।

✓ **BART** Encoder ও decoder উভয়ই আছে। BERT ও GPT-2-এর pretraining scheme একত্রিত করে corrupted input পুনরুদ্ধার করতে শেখে।

✗ **mBART-50** BART-এর multilingual version, ৫০টি ভাষায় pretrained।

কেন **mT5** বেছে নিলাম?

আমরা **mT5** ব্যবহার করব। T5-এ প্রতিটি NLP কাজ একটি **prompt prefix** দিয়ে সম্পন্ন হয় (যেমন `summarize:`), যা মডেলকে prompt অনুযায়ী output তৈরি করতে নির্দেশ দেয়। mT5-এ prefix নেই, কিন্তু T5-এর মতোই versatile এবং multilingual হওয়ার সুবিধা আছে।

Try it out! এই অধ্যায় শেষ করার পর **mT5** এবং **mBART**-এর তুলনা করুন। ***Bonus*** হিসেবে শুধু ইংরেজি রিভিউতে **T5 fine-tune** করার চেষ্টা করুন—তবে সেক্ষেত্রে **preprocessing**-এ **input**-এর শুরুতে `summarize:` যোগ করতে হবে।

ধাপ ৩—ডেটা Preprocessing

Tokenizer লোড করা

আমরা `mt5-small` checkpoint ব্যবহার করব, যাতে সীমিত সময়ে মডেল fine-tune করা সম্ভব হয়:

```
from transformers import AutoTokenizer
```

```
model_checkpoint = "google/mt5-small"
```

```
tokenizer = AutoTokenizer.from_pretrained(model_checkpoint)
```

টিপ: *NLP* প্রজেক্টের শুরুতে সর্বদা ছোট ডেটায় “*small*” মডেল দিয়ে শুরু করুন। এতে দ্রুত *debug* ও *iterate* করা সম্ভব। ফলাফলে সন্তুষ্ট হলে শুধু *model checkpoint* পরিবর্তন করে *scale up* করুন!

একটি ছোট উদাহরণে tokenizer পরীক্ষা করা যাক:

```
inputs = tokenizer("I loved reading the Hunger Games!")
```

```
inputs
```

আউটপুট:

```
{'input_ids': [336, 259, 28387, 11807, 287, 62893, 295, 12507, 1], 'attention_mask': [1, 1, 1, 1, 1, 1, 1, 1, 1]}
```

Input ID-গুলো decode করে দেখি কী ধরনের tokenizer ব্যবহার হচ্ছে:

```
tokenizer.convert_ids_to_tokens(inputs.input_ids)
```

আউটপুট:

```
['_I', ' ', 'loved', ' ', '_reading', ' ', '_the', ' ', '_Hung', 'er', ' ', '_Games', ' ', '</s>']
```

বিশেষ Unicode character `_` এবং end-of-sequence token `</s>` থেকে বোঝা যাচ্ছে এটি **SentencePiece tokenizer**, যা **Unigram segmentation algorithm** ব্যবহার করে। Unigram multilingual corpus-এর জন্য বিশেষভাবে উপযোগী কারণ এটি accents, punctuation এবং জাপানি ভাষার মতো whitespace-বিহীন ভাষাগুলোর ব্যাপারে agnostic।

Preprocess Function

Summarization-এ একটি সূক্ষ্মতা আছে: আমাদের label-গুলোও text, তাই সেগুলো মডেলের maximum context size ছাড়িয়ে যেতে পারে। তাই রিভিউ ও title—উভয়েই truncation প্রয়োগ করতে হবে:

```
max_input_length = 512
```

```
max_target_length = 30
```

```
def preprocess_function(examples):
```

```
    model_inputs = tokenizer(
        examples["review_body"],
        max_length=max_input_length,
        truncation=True,
    )
```

```
    labels = tokenizer(
        examples["review_title"], max_length=max_target_length, truncation=True
```

```
)  
model_inputs["labels"] = labels["input_ids"]  
return model_inputs
```

এই function-এ `max_input_length = 512` রিভিউর জন্য এবং `max_target_length = 30` title-এর জন্য নির্ধারণ করা হয়েছে—কারণ রিভিউ সাধারণত title-এর চেয়ে অনেক বড়।

এখন `Dataset.map()` দিয়ে পুরো corpus tokenize করি:

```
tokenized_datasets = books_dataset.map(preprocess_function, batched=True)
```

টিপ: `batched=True` ব্যবহার করলে default ১০০০ উদাহরণের batch-এ encoding হয় এবং Transformers-এর fast tokenizer-গুলোর multithreading সুবিধা নেওয়া যায়।

ধাপ ৪—ROUGE Score: Summarization Metric

কেন ROUGE?

Text generation task পরিমাপ করা classification-এর মতো সহজ নয়। একটি রিভিউর জন্য অনেক valid summary থাকতে পারে। Generated summary ও reference summary-এর exact match করা যুক্তিযুক্ত নয়।

Summarization-এ সবচেয়ে বহুল ব্যবহৃত metric হলো **ROUGE score** (*Recall-Oriented Understudy for Gisting Evaluation*)। এর মূল ধারণা হলো generated summary-কে মানুষের তৈরি reference summary-এর সাথে তুলনা করা।

Precision ও Recall হিসাব

ধরা যাক:

```
generated_summary = "I absolutely loved reading the Hunger Games"  
reference_summary = "I loved reading the Hunger Games"
```

Recall পরিমাপ করে, reference summary-এর কতটা generated summary-তে ধরা পড়েছে:

$$\text{Recall} = \frac{\text{Number of overlapping words}}{\text{Total number of words in reference summary}}$$

এই উদাহরণে: $6/6 = 1.0$ (perfect recall)।

কিন্তু যদি generated summary হতো “*I really really loved reading the Hunger Games all night*”—এখানেও recall নিখুঁত হতো, কিন্তু summary টি verbose। তাই **Precision** দেখা দরকার:

$$\text{Precision} = \frac{\text{Number of overlapping words}}{\text{Total number of words in generated summary}}$$

Verbose summary-এর precision: $6/10 = 0.6$ —অনেকটা খারাপ।

সংক্ষিপ্ত summary-এর precision: $6/7 = 0.86$ —অনেক ভালো।

ROUGE ইনস্টল ও ব্যবহার

```
!pip install rouge_score
import evaluate
rouge_score = evaluate.load("rouge")
scores = rouge_score.compute(
    predictions=[generated_summary], references=[reference_summary]
)
scores
```

আউটপুট:

```
{'rouge1': AggregateScore(low=Score(precision=0.86, recall=1.0, fmeasure=0.92),
mid=Score(precision=0.86, recall=1.0, fmeasure=0.92), high=Score(precision=0.86, recall=1.0,
fmeasure=0.92)),
'rouge2': AggregateScore(low=Score(precision=0.67, recall=0.8, fmeasure=0.73),
mid=Score(precision=0.67, recall=0.8, fmeasure=0.73), high=Score(precision=0.67, recall=0.8,
fmeasure=0.73)),
'rougeL': AggregateScore(low=Score(precision=0.86, recall=1.0, fmeasure=0.92),
mid=Score(precision=0.86, recall=1.0, fmeasure=0.92), high=Score(precision=0.86, recall=1.0,
fmeasure=0.92)),
'rougeLsum': AggregateScore(low=Score(precision=0.86, recall=1.0, fmeasure=0.92),
mid=Score(precision=0.86, recall=1.0, fmeasure=0.92), high=Score(precision=0.86, recall=1.0,
fmeasure=0.92))}
```

এই output-এ অনেক তথ্য! এগুলো বোঝা যাক:

- Datasets precision, recall ও F1-score-এর জন্য **confidence interval** compute করে—**low**, **mid**, **high** দিয়ে দেখানো হয়।
- **rouge1**—unigram overlap (এককভাবে শব্দের মিল)
- **rouge2**—bigram overlap (শব্দজোড়ার মিল)
- **rougeL**—সবচেয়ে দীর্ঘ মিলে যাওয়া শব্দের ধারা (longest common substring)
- **rougeLsum**—পুরো summary জুড়ে rougeL গণনা

mid score দেখি:

```
scores["rouge1"].mid
```

আউটপুট:

```
Score(precision=0.86, recall=1.0, fmeasure=0.92)
```

সংখ্যাগুলো আমাদের হিসাবের সাথে মিলে যাচ্ছে।

ধাপ ৫—শক্তিশালী **Baseline** তৈরি

Lead-3 Baseline

Text summarization-এর একটি সাধারণ baseline হলো একটি article-এর প্রথম তিনটি বাক্য নেওয়া—এটাকে **lead-3 baseline** বলা হয়।

সাধারণ full stop দিয়ে বাক্য শনাক্ত করলে “U.S.” বা “U.N.” এর মতো abbreviation-এ সমস্যা হয়। তাই আমরা **nltk** library ব্যবহার করব:

```
!pip install nltk
import nltk
nltk.download("punkt")
from nltk.tokenize import sent_tokenize

def three_sentence_summary(text):
    return "\n".join(sent_tokenize(text)[:3])

print(three_sentence_summary(books_dataset["train"][1]["review_body"]))
```

আউটপুট:

```
'I grew up reading Koontz, and years ago, I stopped,convinced i had "outgrown" him.'
'Still,when a friend was looking for something suspenseful too read, I suggested Koontz.'
'She found Strangers.'
```

এখন validation set-এ ROUGE score গণনা করি:

```
def evaluate_baseline(dataset, metric):
    summaries = [three_sentence_summary(text) for text in dataset["review_body"]]
    return metric.compute(predictions=summaries, references=dataset["review_title"])
import pandas as pd
score = evaluate_baseline(books_dataset["validation"], rouge_score)
rouge_names = ["rouge1", "rouge2", "rougeL", "rougeLsum"]
rouge_dict = dict((rn, round(score[rn].mid.fmeasure * 100, 2)) for rn in rouge_names)
rouge_dict
```

আউটপুট:

```
{'rouge1': 16.74, 'rouge2': 8.83, 'rougeL': 15.6, 'rougeLsum': 15.96}
```

rouge2 score অনেক কম—কারণ review title সাধারণত সংক্ষিপ্ত, আর lead-3 baseline অনেক বেশি verbose।

ধাপ ৬—Trainer API দিয়ে mT5 Fine-Tuning

মডেল লোড

```
from transformers import AutoModelForSeq2SeqLM
model = AutoModelForSeq2SeqLM.from_pretrained(model_checkpoint)
Sequence-to-sequence task-এ আমরা network-এর সব weight ধরে রাখি। Text classification-এ যেমন head পরিবর্তন করতে হয়েছিল, এখানে সেটা দরকার নেই।
```

Hugging Face Hub Login

```
from huggingface_hub import notebook_login
notebook_login()
```

অথবা terminal-এ:

```
huggingface-cli login
```

Training Arguments নির্ধারণ

```
from transformers import Seq2SeqTrainingArguments
batch_size = 8
num_train_epochs = 8
# Show the training loss with every epoch
logging_steps = len(tokenized_datasets["train"]) // batch_size
model_name = model_checkpoint.split("/")[-1]
args = Seq2SeqTrainingArguments(
    output_dir=f"{model_name}-finetuned-amazon-en-es",
    evaluation_strategy="epoch",
    learning_rate=5.6e-5,
    per_device_train_batch_size=batch_size,
    per_device_eval_batch_size=batch_size,
    weight_decay=0.01,
    save_total_limit=3,
    num_train_epochs=num_train_epochs,
    predict_with_generate=True,
    logging_steps=logging_steps,
    push_to_hub=True,
```

)

এখানে `predict_with_generate=True` নির্ধারণ করা হয়েছে যাতে evaluation-এ প্রতিটি epoch-এ summary generate করে ROUGE score গণনা করা যায়। `save_total_limit=3` দিয়ে সর্বোচ্চ ৩টি checkpoint সংরক্ষণ করা হবে—কারণ mT5-এর "small" version-ও প্রায় ১ GB জায়গা নেয়।

`push_to_hub=True` ট্রেনিং শেষে মডেল Hub-এ upload করবে।

Compute Metrics Function

Training চলাকালীন ROUGE score গণনা করার জন্য `compute_metrics()` function প্রয়োজন। এখানে predicted ও reference labels decode করে text-এ রূপান্তর করতে হবে:

```
import numpy as np
```

```
def compute_metrics(eval_pred):
    predictions, labels = eval_pred
    # Decode generated summaries into text
    decoded_preds = tokenizer.batch_decode(predictions, skip_special_tokens=True)
    # Replace -100 in the labels as we can't decode them
    labels = np.where(labels != -100, labels, tokenizer.pad_token_id)
    # Decode reference summaries into text
    decoded_labels = tokenizer.batch_decode(labels, skip_special_tokens=True)
    # ROUGE expects a newline after each sentence
    decoded_preds = ["\n".join(sent_tokenize(pred.strip())) for pred in decoded_preds]
    decoded_labels = ["\n".join(sent_tokenize(label.strip())) for label in decoded_labels]
    # Compute ROUGE scores
    result = rouge_score.compute(
        predictions=decoded_preds, references=decoded_labels, use_stemmer=True
    )
    # Extract the median scores
    result = {key: value.mid.fmeasure * 100 for key, value in result.items()}
    return {k: round(v, 4) for k, v in result.items()}
```

Data Collator

mT5 encoder-decoder Transformer, তাই batch প্রস্তুতিতে একটি সূক্ষ্মতা আছে: decoding-এর সময় labels-কে ডানদিকে এক ধাপ **shift** করতে হয়। এটি নিশ্চিত করে যে decoder শুধু আগের ground truth labels দেখতে পায়, বর্তমান বা পরেরটা নয়।

Transformers-এর `DataCollatorForSeq2Seq` এটি স্বয়ংক্রিয়ভাবে করে:

```
from transformers import Seq2SeqTrainer
trainer = Seq2SeqTrainer(
```

```
model,
args,
train_dataset=tokenized_datasets["train"],
eval_dataset=tokenized_datasets["validation"],
data_collator=data_collator,
tokenizer=tokenizer,
compute_metrics=compute_metrics,
)
trainer.train()
```

Training চলাকালীন প্রতিটি epoch-এ training loss কমে এবং ROUGE score বাড়ে দেখা যাবে।

Final Evaluation

```
trainer.evaluate()
```

আউটপুট:

```
{'eval_loss': 3.028524398803711,
'eval_rouge1': 16.9728,
'eval_rouge2': 8.2969,
'eval_rougeL': 16.8366,
'eval_rougeLsum': 16.851,
'eval_gen_len': 10.1597,
'eval_runtime': 6.1054,
'eval_samples_per_second': 38.982,
'eval_steps_per_second': 4.914}
```

আমাদের মডেল lead-3 baseline-কে স্পষ্টভাবে ছাড়িয়ে গেছে!

Hub-এ Push করা

```
trainer.push_to_hub(commit_message="Training complete", tags="summarization")
```

আউটপুট:

```
'https://huggingface.co/huggingface-course/mt5-finetuned-amazon-en-es/commit/aa0536b829b28e73e1e4b94b8a5aacec420d40e0'
```

`tags="summarization"` argument নিশ্চিত করে যে Hub-এ widget টি summarization pipeline হিসেবে দেখাবে।

সবকিছু প্রস্তুত করা

Accelerate দিয়ে fine-tuning অনেকটা Chapter 3-এর text classification উদাহরণের মতো। মূল পার্থক্য হলো training-এ explicitly summary generate করতে হবে এবং ROUGE score গণনা করতে হবে।

প্রথমে dataset-এর format “torch” করা যাক:

```
tokenized_datasets.set_format("torch")
```

মডেল নতুন করে লোড করি:

```
model = AutoModelForSeq2SeqLM.from_pretrained(model_checkpoint)
```

Data collator ও DataLoader তৈরি:

```
from torch.utils.data import DataLoader
batch_size = 8
train_dataloader = DataLoader(
    tokenized_datasets["train"],
    shuffle=True,
    collate_fn=data_collator,
    batch_size=batch_size,
)
eval_dataloader = DataLoader(
    tokenized_datasets["validation"], collate_fn=data_collator, batch_size=batch_size
)
```

Optimizer নির্ধারণ:

```
from torch.optim import AdamW
optimizer = AdamW(model.parameters(), lr=2e-5)
```

Accelerator দিয়ে সব prepare করা:

```
from accelerate import Accelerator
accelerator = Accelerator()
model, optimizer, train_dataloader, eval_dataloader = accelerator.prepare(
    model, optimizer, train_dataloader, eval_dataloader
)
```

TPU-তে training করলে উপরের সব code একটি dedicated training function-এ রাখতে হবে।

Learning Rate Schedule ও Post-processing

Linear learning rate schedule:

```

from transformers import get_scheduler
num_train_epochs = 10
num_update_steps_per_epoch = len(train_dataloader)
num_training_steps = num_train_epochs * num_update_steps_per_epoch
lr_scheduler = get_scheduler(
    "linear",
    optimizer=optimizer,
    num_warmup_steps=0,
    num_training_steps=num_training_steps,
)

```

ROUGE metric-এর জন্য generated summary-গুলো newline দিয়ে বাক্যে ভাগ করতে হবে:

```

def postprocess_text(preds, labels):
    preds = [pred.strip() for pred in preds]
    labels = [label.strip() for label in labels]
    # ROUGE expects a newline after each sentence
    preds = ["\n".join(nltk.sent_tokenize(pred)) for pred in preds]
    labels = ["\n".join(nltk.sent_tokenize(label)) for label in labels]
    return preds, labels

```

Hub Repository তৈরি

```

from huggingface_hub import get_full_repo_name
model_name = "test-bert-finetuned-squad-accelerate"
repo_name = get_full_repo_name(model_name)
repo_name

```

আউটপুট:

```

'lewtun/mt5-finetuned-amazon-en-es-accelerate'
from huggingface_hub import Repository
output_dir = "results-mt5-finetuned-squad-accelerate"
repo = Repository(output_dir, clone_from=repo_name)

```

Training Loop

Training loop মূলত চারটি ধাপে বিভক্ত:

1. প্রতিটি epoch-এ `train_dataloader`-এর সব উদাহরণ দিয়ে মডেল ট্রেন করা
2. প্রতিটি epoch শেষে token generate করে summary decode করা
3. ROUGE score গণনা করা
4. Checkpoint সংরক্ষণ করে Hub-এ push করা (`blocking=False` দিয়ে asynchronously, যাতে ট্রেনিং থামাতে না হয়)


```

from tqdm.auto import tqdm
import torch
import numpy as np
progress_bar = tqdm(range(num_training_steps))
for epoch in range(num_train_epochs):
    # Training
    model.train()
    for step, batch in enumerate(train_dataloader):
        outputs = model(**batch)
        loss = outputs.loss
        accelerator.backward(loss)
        optimizer.step()
        lr_scheduler.step()
        optimizer.zero_grad()
        progress_bar.update(1)
    # Evaluation
    model.eval()
    for step, batch in enumerate(eval_dataloader):
        with torch.no_grad():
            generated_tokens = accelerator.unwrap_model(model).generate(
                batch["input_ids"],
                attention_mask=batch["attention_mask"],
            )
            generated_tokens = accelerator.pad_across_processes(
                generated_tokens, dim=1, pad_index=tokenizer.pad_token_id
            )
            labels = batch["labels"]
            # If we did not pad to max length, we need to pad the labels too
            labels = accelerator.pad_across_processes(
                batch["labels"], dim=1, pad_index=tokenizer.pad_token_id
            )
            generated_tokens = accelerator.gather(generated_tokens).cpu().numpy()
            labels = accelerator.gather(labels).cpu().numpy()
            # Replace -100 in the labels as we can't decode them
            labels = np.where(labels != -100, labels, tokenizer.pad_token_id)
            if isinstance(generated_tokens, tuple):
                generated_tokens = generated_tokens[0]
            decoded_preds = tokenizer.batch_decode(
                generated_tokens, skip_special_tokens=True
            )
            decoded_labels = tokenizer.batch_decode(labels, skip_special_tokens=True)
            decoded_preds, decoded_labels = postprocess_text(
                decoded_preds, decoded_labels
            )

```

```

        rouge_score.add_batch(predictions=decoded_preds, references=decoded_labels)
# Compute metrics
result = rouge_score.compute()
# Extract the median ROUGE scores
result = {key: value.mid.fmeasure * 100 for key, value in result.items()}
result = {k: round(v, 4) for k, v in result.items()}
print(f"Epoch {epoch}:", result)
# Save and upload
accelerator.wait_for_everyone()
unwrapped_model = accelerator.unwrap_model(model)
unwrapped_model.save_pretrained(output_dir, save_function=accelerator.save)
if accelerator.is_main_process:
    tokenizer.save_pretrained(output_dir)
    repo.push_to_hub(
        commit_message=f"Training in progress epoch {epoch}", blocking=False
    )

```

আউটপুট:

```

Epoch 0: {'rouge1': 5.6351, 'rouge2': 1.1625, 'rougeL': 5.4866, 'rougeLsum': 5.5005}
Epoch 1: {'rouge1': 9.8646, 'rouge2': 3.4106, 'rougeL': 9.9439, 'rougeLsum': 9.9306}
Epoch 2: {'rouge1': 11.0872, 'rouge2': 3.3273, 'rougeL': 11.0508, 'rougeLsum': 10.9468}
Epoch 3: {'rouge1': 11.8587, 'rouge2': 4.8167, 'rougeL': 11.7986, 'rougeLsum': 11.7518}
Epoch 4: {'rouge1': 12.9842, 'rouge2': 5.5887, 'rougeL': 12.7546, 'rougeLsum': 12.7029}
Epoch 5: {'rouge1': 13.4628, 'rouge2': 6.4598, 'rougeL': 13.312, 'rougeLsum': 13.2913}
Epoch 6: {'rouge1': 12.9131, 'rouge2': 5.8914, 'rougeL': 12.6896, 'rougeLsum': 12.5701}
Epoch 7: {'rouge1': 13.3079, 'rouge2': 6.2994, 'rougeL': 13.1536, 'rougeLsum': 13.1194}
Epoch 8: {'rouge1': 13.96, 'rouge2': 6.5998, 'rougeL': 13.9123, 'rougeLsum': 13.7744}
Epoch 9: {'rouge1': 14.1192, 'rouge2': 7.0059, 'rougeL': 14.1172, 'rougeLsum': 13.9509}

```

দারুণ! Trainer ব্যবহার করে পাওয়া ফলাফলের সাথে এটি প্রায় একই।

ধাপ ৮—**Fine-tuned** মডেল ব্যবহার করা

Hub-এ push করার পর inference widget বা **pipeline** object দিয়ে মডেল ব্যবহার করা যাবে:

```

from transformers import pipeline
hub_model_id = "huggingface-course/mt5-small-finetuned-amazon-en-es"
summarizer = pipeline("summarization", model=hub_model_id)

```

Test set থেকে উদাহরণ দেখার জন্য একটি function:

```

def print_summary(idx):

```

```
review = books_dataset["test"][idx]["review_body"]
title = books_dataset["test"][idx]["review_title"]
summary = summarizer(books_dataset["test"][idx]["review_body"])[0]["summary_text"]
print(f">>> Review: {review}")
print(f">>> Title: {title}")
print(f">>> Summary: {summary}")
```

ইংরেজি রিভিউর উদাহরণ

```
print_summary(100)
```

আউটপুট:

```
'>>> Review: Nothing special at all about this product... the book is too small and stiff and hard
to write in. The huge sticker on the back doesn't come off and looks super tacky. I would not
purchase this again. I could have just bought a journal from the dollar store and it would be
basically the same thing. It's also really expensive for what it is.'
'>>> Title: Not impressed at all... buy something else'
'>>> Summary: Nothing special at all about this product'
```

মডেলটি রিভিউর অংশ থেকে **abstractive summarization** করতে পারছে।

স্প্যানিশ রিভিউর উদাহরণ

```
print_summary(0)
```

আউটপুট:

```
'>>> Review: Es una trilogia que se hace muy facil de leer. Me ha gustado, no me esperaba el
final para nada'
'>>> Title: Buena literatura para adolescentes'
'>>> Summary: Muy facil de leer'
```

Summary-টির ইংরেজি অর্থ: “*Very easy to read*”—যা সরাসরি রিভিউ থেকে নেওয়া হয়েছে। এটি আমাদের mT5 মডেলের **multilingual** সক্ষমতার সুন্দর প্রদর্শন!

কী শিখলাম?

বিষয় মূল কথা **Corpus** Multilingual Amazon Reviews থেকে ইংরেজি ও স্প্যানিশ book reviews ব্যবহার মডেল mT5-small—multilingual, encoder-decoder architecture **Tokenizer** SentencePiece (Unigram)—multilingual corpus-এর জন্য আদর্শ **Metric** ROUGE score—recall, precision ও F1 সমন্বয়ে **Baseline** Lead-3—প্রথম ৩ বাক্য নেওয়া **Fine-tuning** Seq2SeqTrainer বা Accelerate—দুটোই সমান কার্যকর

Thank you for taking the time to go through this notebook. I hope you found it helpful and informative.

If you liked this work, please consider giving it an upvote. Your support motivates me to create more useful content.

You can also connect with me and explore more of my work here:

Full Github Repo : <https://github.com/Sakhawathossen04/Building-LLMs-from-scratch-In-Bangla>

GitHub: <https://github.com/Sakhawathossen04>

LinkedIn: <https://www.linkedin.com/in/sakhawathossenofficial/>

Kaggle : <https://www.kaggle.com/sakhawathossen>

Feel free to share your feedback or suggestions. Your input is always appreciated.